SIMATS ENGINEERING

ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.* (BL4)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 30**
Question Count: 5

Code of Conduct

I Atmakuri Sai Mrudula/192372087 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: A. Sai Mrudula

Assessment Tasks

Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.
2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.
3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.
4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed

messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.

5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

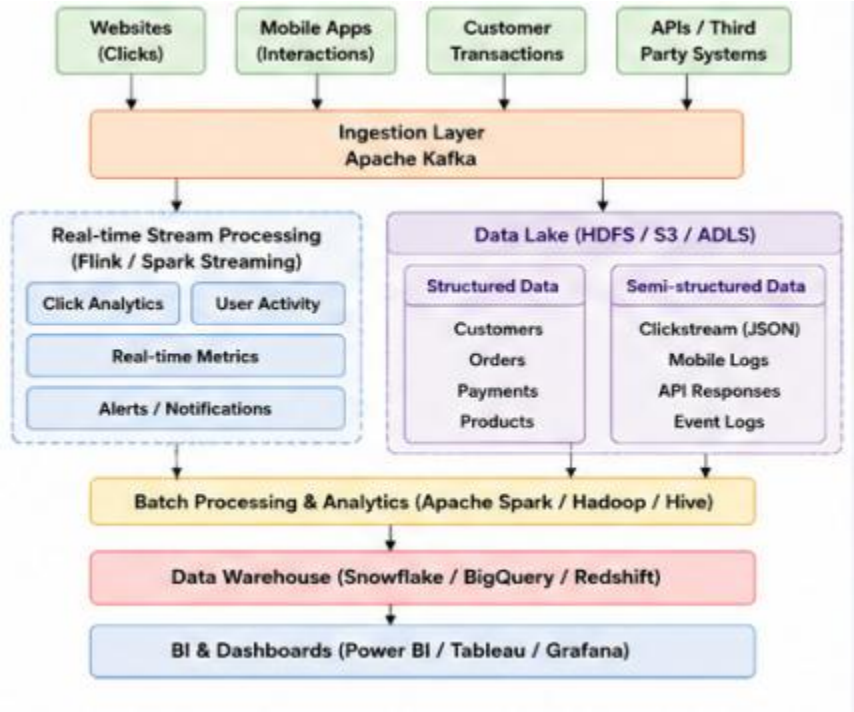
Reverse Engineering Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference
Understanding of Architecture	25%	Shows clear understanding of underlying big data architecture	Good understanding	Partial understanding	Poor understanding
Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence
Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis
Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.

A big data platform processing billions of clicks, mobile interactions, and transactions requires a distributed architecture. Based on the workload, the probable architecture would use distributed storage, event streaming, batch processing, and real-time analytics.

Probable Architecture



1. Distributed Storage

The probable distributed storage system is:

- Hadoop HDFS for on-premise environments
- Amazon S3 / Azure Data Lake / Google Cloud Storage for cloud environments

The data is distributed across multiple nodes. Replication ensures that data remains available even when individual machines fail.

2. Data Ingestion

Apache Kafka is a likely ingestion framework because billions of events are generated continuously.

3. Structured and Semi-Structured Data

Structured data includes:

- Customer records
- Orders
- Payments
- Product information

This can be stored in relational databases and data warehouses.

Semi-structured data includes:

- JSON clickstream events
- Mobile application logs
- API responses
- Web events

These can be stored in a data lake in their original format.

4. Indexing

For fast analysis, the system may use:

- Elasticsearch/OpenSearch for event searching
- Database indexes for structured records
- Partitioning by date, region, customer, or event type
- Columnar formats such as Parquet

This reduces the amount of data that needs to be scanned.

5. Batch Processing

Historical data can be processed using:

- Apache Spark
- Hadoop MapReduce
- Hive

Spark is particularly suitable for large-scale analytics and machine learning.

6. Stream Processing

For real-time analytics, probable technologies include:

- Apache Flink
- Spark Structured Streaming
- Kafka Streams

7. Scalability and Partitioning

Kafka partitions events across multiple brokers.

Similarly, distributed storage partitions data across multiple nodes.

For example:

Topic

├── Partition 1 → Node 1

├── Partition 2 → Node 2

├── Partition 3 → Node 3

└── Partition 4 → Node 4

As workload increases, additional partitions and processing nodes can be added.

8. BI and Dashboards

Processed data can be sent to:

- Snowflake
- BigQuery
- Redshift
- ClickHouse

Business intelligence tools such as Power BI, Tableau, or Grafana can connect to these analytical stores.

The probable platform uses Kafka for ingestion, HDFS/S3 for distributed storage, Spark/Flink for processing, partitioned data lakes for scalability, and BI tools for visualization. This architecture supports both large-scale historical analysis and real-time reporting.

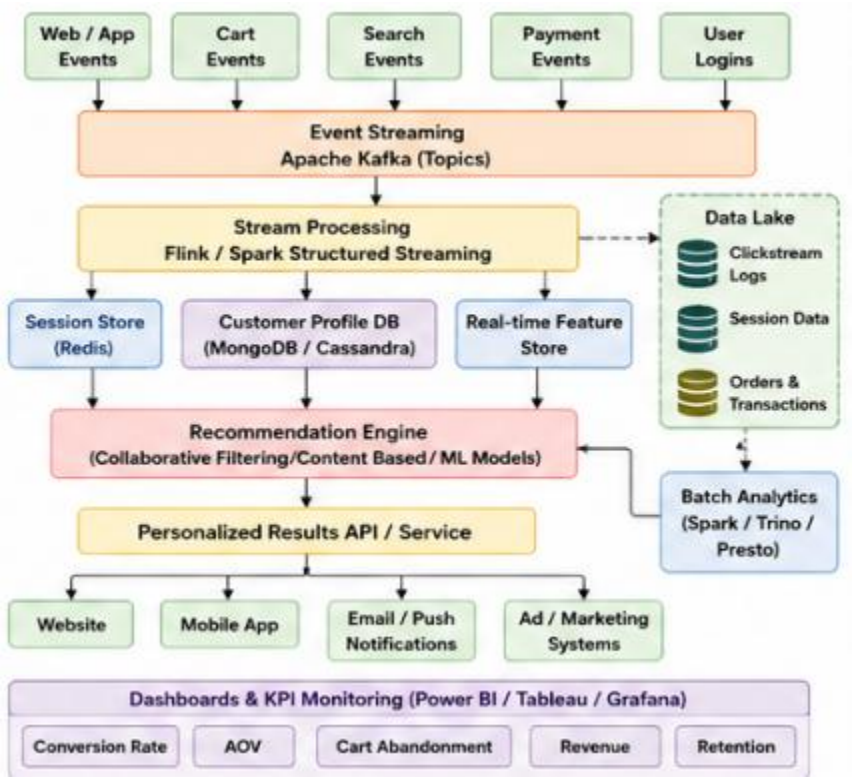
2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.

The platform generates several types of events:

- Product views
- Cart additions
- Cart abandonment
- Checkout
- Payment
- Search
- Customer login

The system needs to combine these events with customer profiles and transaction data to provide real-time personalization.

Probable Architecture



1. Event Streaming

Apache Kafka is a probable event streaming platform.
Different topics can represent:
product-view
cart-add
checkout
payment
search

user-login
This allows different consumers to process events independently.

2. Customer Profiling
A customer profile may contain:
Customer ID
Age group
Location
Previous purchases
Favorite categories
Average order value
Browsing history
Probable technologies include:

- Cassandra
- MongoDB
- DynamoDB
- Redis for frequently accessed information

3. Session Management
Session data tracks a customer's current activity.
The streaming system can immediately identify this as an abandoned or active shopping session.

4. Synchronizing Different Data
The platform uses a common **Customer ID** or session ID.
Clickstream
↓
Customer ID
↓
Customer Profile
↓
Order History
↓
Recommendation
This allows structured order data to be combined with semi-structured browsing events.

5. Caching
Redis is a probable caching system.
It can store:

- Popular products
- Customer preferences
- Active sessions
- Recommendation results

Caching reduces database access time and improves response latency.

6. Distributed Query Engines
Large datasets may be queried using:

- Trino
- Presto
- Spark SQL
- BigQuery

These systems allow analysts to query large datasets without moving all data into one centralized database.

7. Recommendation Workflow

Customer Behavior
↓
Feature Engineering
↓
ML Model
↓
Recommendation Score
↓
Ranking
↓
Top Products
↓
Customer

Possible models:

- Collaborative filtering
- Content-based filtering
- Logistic regression
- Random Forest
- XGBoost
- Neural networks

8. Scalability and Fault Tolerance

Scalability can be achieved through:

- Kafka partitions
- Distributed databases
- Horizontal scaling
- Cloud auto-scaling
- Data replication

Fault tolerance can be achieved through:

- Kafka replication
- Database replicas
- Backup systems
- Multiple processing nodes

9. KPI Monitoring

Important KPIs include:

- Conversion rate
- Cart abandonment rate
- Average order value
- Customer retention
- Revenue
- Product views
- Recommendation click-through rate

The probable architecture combines Kafka, Flink/Spark, Redis, distributed databases, ML recommendation models, and BI dashboards to provide real-time personalization while maintaining scalability and fault tolerance.

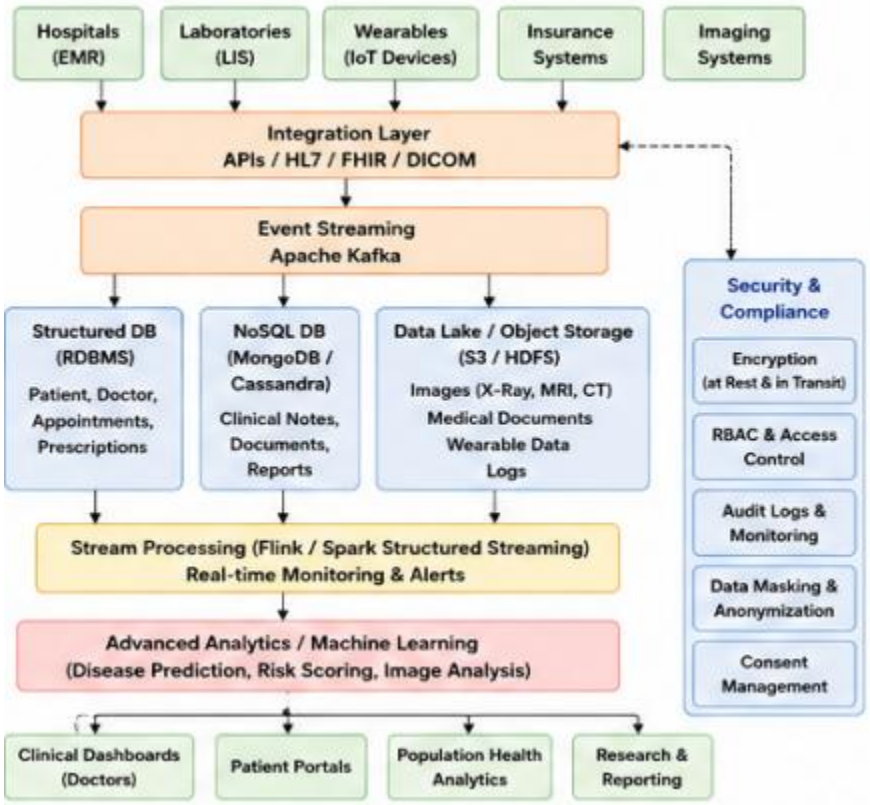
3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.

Healthcare data is highly diverse:

- Patient histories
- Diagnostic images
- Wearable sensor data
- Prescriptions
- Laboratory results
- Hospital records

The system therefore requires a hybrid storage architecture capable of handling structured, semi-structured, and unstructured data.

Probable Architecture



1. Hybrid Database Architecture

Relational databases can store structured data:

- Patient ID
- Doctor ID
- Prescription
- Appointment
- Lab result

Examples:

- PostgreSQL
- MySQL
- Oracle

NoSQL databases can store flexible healthcare records:
- MongoDB
- Cassandra

Object storage/data lakes can store:
- X-rays
- MRI scans
- CT images
- Medical documents
- Audio/video records

2. Data Integration

Healthcare organizations use interoperability technologies such as:
- HL7
- FHIR
- DICOM for medical images
- APIs

These enable hospitals, laboratories, and insurance systems to exchange information.

3. Streaming Framework

Wearable devices continuously produce:
Heart Rate
Blood Pressure
Oxygen Level
Temperature
Activity
Kafka can collect these events, while Flink or Spark Streaming can process them.

4. Machine Learning

ML models can support:
- Disease prediction
- Patient risk scoring
- Readmission prediction
- Abnormal vital-sign detection
- Medical image classification

Example:
Patient Data
↓
Feature Extraction
↓
ML Model
↓
Risk Score
↓

Doctor Alert

5. Privacy and Security

Healthcare information is highly sensitive.
Security mechanisms include:

- Encryption at rest
- Encryption in transit
- Role-Based Access Control
- Multi-factor authentication
- Data masking
- Audit logs
- Patient consent management

6. Analytics Pipeline

Patient Data
↓
Data Integration
↓
Data Cleaning
↓
Data Lake
↓
Spark Processing
↓
Analytics / ML
↓
Clinical Dashboard

7. Population Health

Aggregated and appropriately protected data can be analyzed to identify:

- Disease trends
- Regional health patterns
- Hospital utilization
- High-risk populations
- Treatment outcomes

A healthcare big data platform would likely combine SQL/NoSQL databases, cloud object storage, Kafka, Spark/Flink, HL7/FHIR/DICOM interoperability, and machine learning while applying strong encryption, access control, auditing, and privacy protections.

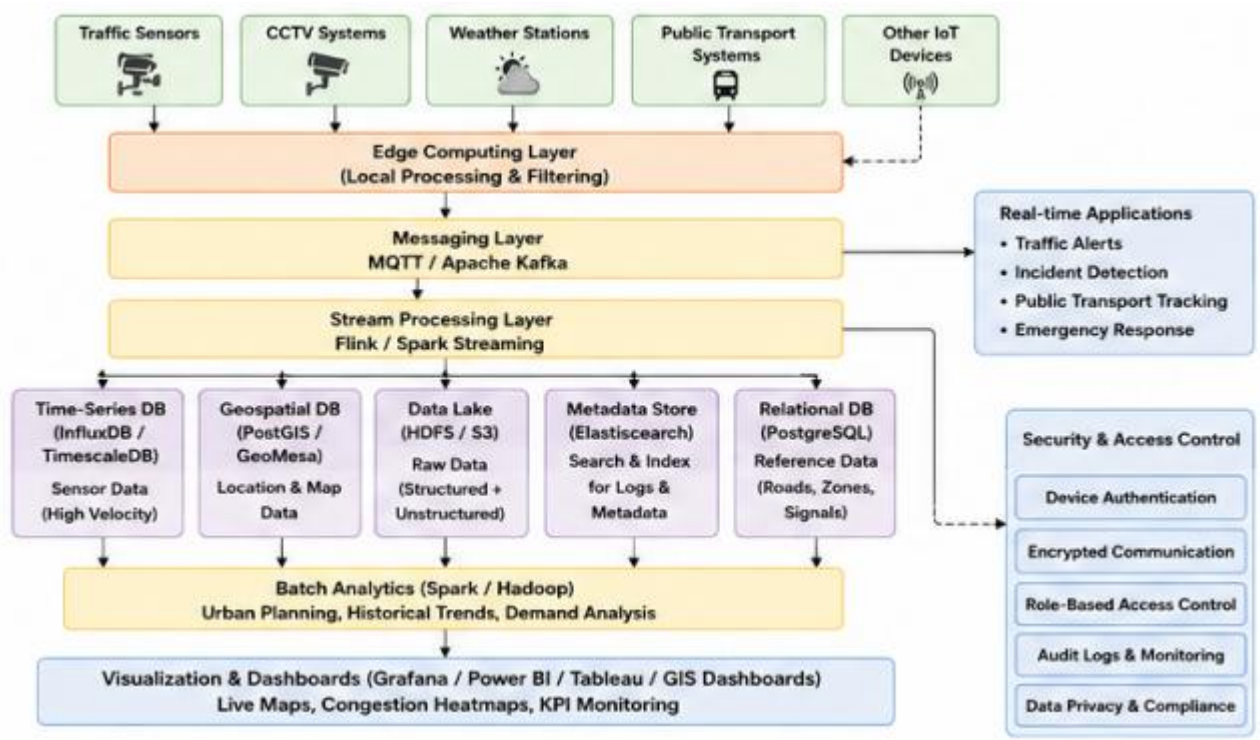
4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.

A smart city continuously generates information from:

- Traffic sensors
- CCTV metadata
- Weather stations
- Public transportation
- GPS systems
- Road infrastructure

The platform must provide real-time operational intelligence while also supporting long-term urban planning.

Probable Architecture



1. Edge Computing

Instead of sending all raw data to the central cloud, edge devices can process some information locally.
For example:
Traffic Camera
↓
Edge Device
↓
Vehicle Count

↓
Central Platform
This reduces network bandwidth and latency.

2. Messaging

MQTT is suitable for lightweight IoT communication.
Kafka can be used for large-scale event streaming and durable event pipelines.

3. Centralized Storage

A data lake can store historical:

- Traffic information
- Weather data
- Transportation data
- CCTV metadata

4. Time-Series Databases

Traffic and sensor data contains timestamps, making time-series databases suitable.
Examples:

- InfluxDB
- TimescaleDB

Example query:
Average traffic volume
between 8:00 AM and 9:00 AM

5. Geospatial Databases

Location-based information can be stored and queried using technologies such as:

- PostgreSQL + PostGIS
- Elasticsearch/OpenSearch geospatial capabilities

They can answer questions such as:
Find traffic congestion within 5 km
of a particular location.

6. Real-Time Congestion Prediction

Traffic Data
+
Weather
+
Historical Traffic
+
Public Transport
↓
Stream Processing
↓
ML Model
↓
Congestion Prediction

↓
Traffic Control Center

7. Batch + Real-Time Analytics

Real-time analytics supports:

- Traffic alerts
- Accident detection
- Bus tracking
- Emergency response

Batch analytics supports:

- Road planning
- Infrastructure planning
- Traffic pattern analysis
- Long-term transportation policies

8. Security

Security mechanisms include:

- Encryption
- Device authentication
- API authentication
- Role-Based Access Control
- Network segmentation
- Security monitoring
- Audit logging

9. Visualization

Live operational dashboards can use:

- Grafana
- Power BI
- Tableau
- GIS dashboards

They can display:

- Traffic maps
- Congestion levels
- Bus locations
- Weather conditions
- Emergency alerts

A smart city platform would likely use edge computing, MQTT/Kafka, Flink/Spark, time-series databases, geospatial databases, data lakes, and real-time dashboards to provide immediate operational intelligence and long-term urban planning insights.

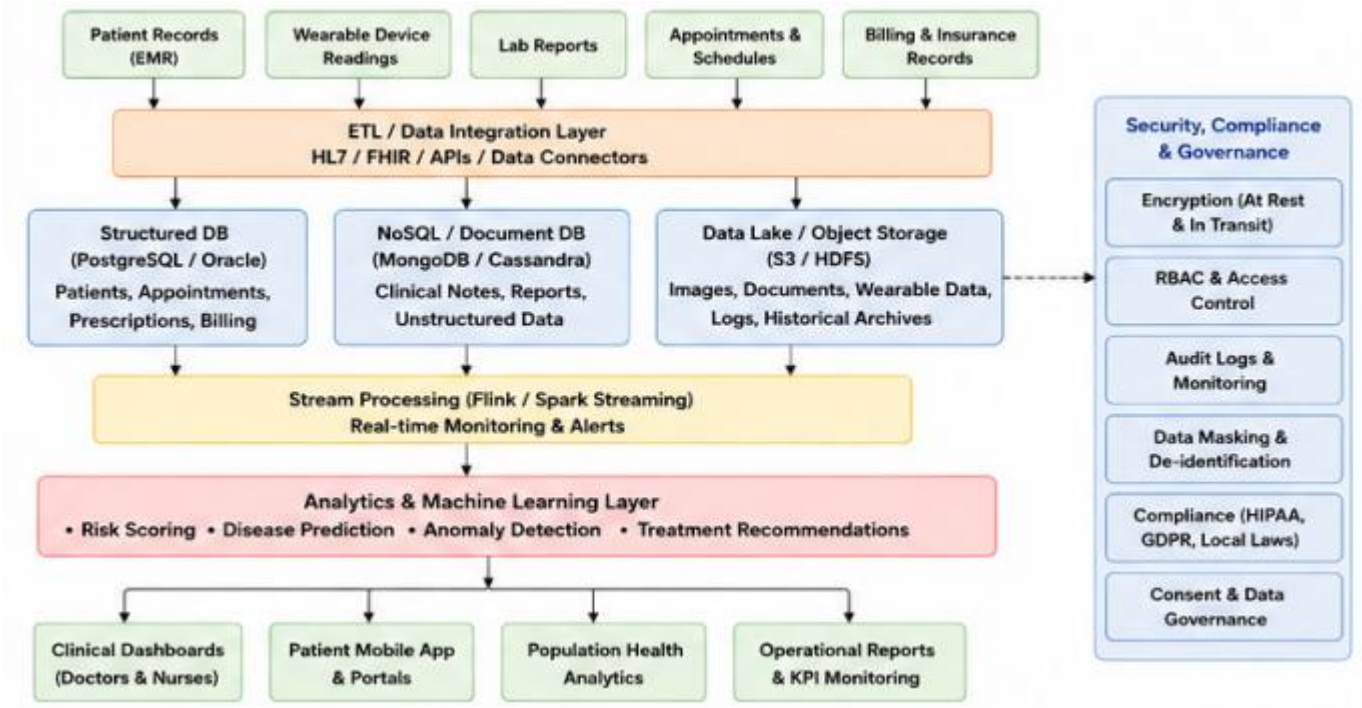
5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

This healthcare analytics platform combines:

- Patient records
- Wearable device readings
- Laboratory reports
- Appointment histories

The major challenges are data heterogeneity, privacy, real-time monitoring, interoperability, and large-scale analytics.

Probable Architecture



1. Hybrid Storage

Structured Data

Patient records, appointments, prescriptions, and lab results can be stored in relational databases.

Examples:

- PostgreSQL
- MySQL
- Oracle

Semi-Structured Data

FHIR/JSON-based healthcare records can be stored using:

- MongoDB
- Document databases

Unstructured Data

Medical documents and diagnostic images can be stored in:

- Cloud object storage
- Data lakes
- HDFS

2. ETL Pipeline

The ETL process includes:

Extract

↓

Transform

↓

Validate

↓

Clean

↓

Load

Data is extracted from hospitals and clinics, transformed into common formats, validated, and loaded into analytical storage.

3. Interoperability

Standards such as HL7 and FHIR can enable healthcare applications to exchange patient information.

For diagnostic images, DICOM can be used.

4. Real-Time Monitoring

Wearable devices continuously send health information.

Wearable

↓

IoT Gateway

↓

Kafka

↓

Flink

↓

Abnormal Condition?

↓

Yes → Alert

For example, an abnormal heart-rate pattern can generate an alert for appropriate clinical review.

5. Distributed Processing

Apache Spark can process millions of patient records simultaneously.

It can support:

- Population health analysis
- Disease trend analysis
- Treatment outcome analysis
- Hospital performance analysis
- Risk-group identification

6. Machine Learning

ML can support:

Diagnosis Risk Scoring

Patient History

+

Lab Results

+

Vital Signs

+

Wearable Data

↓

ML Model

↓

Disease Risk Score

Possible models:

- Logistic Regression
- Random Forest
- XGBoost
- Neural Networks

7. Treatment Recommendations

Models can analyze historical outcomes and patient characteristics to assist qualified healthcare professionals in evaluating potentially suitable treatment options.

8. Encryption and Compliance

The platform should implement:

- Encryption at rest
- Encryption in transit
- Strong authentication
- Role-Based Access Control
- Data masking
- Audit trails
- Consent management
- Data retention policies

Compliance must be designed according to the applicable healthcare and privacy regulations in the jurisdictions where the system operates.

9. Audit Mechanism

Every sensitive data access should be logged:

User → Patient Record → Timestamp → Action

This helps detect unauthorized access and supports accountability.

A healthcare analytics platform should combine hybrid storage, HL7/FHIR interoperability, Kafka/Flink for real-time monitoring, Spark for distributed population analytics, and machine learning for risk assessment, supported by strong encryption, access control, auditing, and privacy governance.